



天数智芯
Iluvatar CoreX

天数智芯

LLaMA-Factory 框架功能说明

版本：V4.3.0-BI150 (x86)|V4.3.6-BI150 (ARM)

日期：2025.6.30

适用产品：天垓 150

声明

版权与保密声明

本文档（属未发表作品）版权归属于天数智芯全权所有，且本声明不构成对本文档已发表的承认。本文档所载全部信息均为天数智芯保密信息（包括但不限于天数智芯技术信息、经营信息、专属于接收方的未公开资料等）。未经天数智芯事先书面许可，严禁任何形式的使用、复制、传播、转录或翻译。

免责声明

天数智芯可以随时对本文档或本文档中描述的产品进行改进和/或更改。本文档包括与天数智芯产品有关的信息，作为说明典型应用的一种方式，因此，不一定提供足以进行生产设计的完整信息。对于本文档中内容的准确性或完整性，天数智芯不做任何陈述或保证。

联系方式

地址：上海闵行区陈行公路 2168 号 3 幢

电话：021-68886607

网址：www.iluvatar.com

目录

1 LLaMA-Factory 框架功能说明	4
1.1 修订记录	4
1.2 LLaMA-Factory 框架概览	4
1.3 支持的特性	4
1.4 安装天数智芯适配版 LLaMA-Factory	5
1.4.1 通过 Docker 镜像安装 LLaMA-Factory	6
1.4.2 在宿主机上安装 LLaMA-Factory	6
1.5 LLaMA-Factory 使用示例	6
1.6 常见问题	9

1 LLaMA-Factory 框架功能说明

1.1 修订记录

- COREX01-BI430-RF12-00: 2025/6/30
 - 更新天数智芯适配版 LLaMA-Factory 的版本为 v0.9.2
 - 新增对 DeepSeek 系列模型的预训练支持
 - 新增 LoRA 微调优化方式
 - 更新通过 Docker 镜像安装 LLaMA-Factory，新增依赖天数智芯适配版 transformers
 - 更新 LLaMA-Factory 使用示例 中的代码
- COREX01-BI420-RF12-00: 2025/1/24
文档首次对外发布

1.2 LLaMA-Factory 框架概览

天数智算软件栈提供天数智芯适配版的 LLaMA-Factory v0.9.2 大模型训练框架。

LLaMA-Factory 是一个简单易用且高效的大型语言模型 (Large Language Model) 训练与微调平台。通过 LLaMA-Factory，您可以在无需编写任何代码的前提下，在本地完成上百种预训练模型的微调。

1.3 支持的特性

天数智芯适配版的 LLaMA-Factory 大模型训练框架功能有：

- 支持预训练的模型种类包括但不限于：
 - LLaMA [1]
 - LLaVA
 - Mistral
 - Mixtral-MoE
 - Qwen [1]
 - Yi
 - Gemma
 - Baichuan
 - ChatGLM
 - Phi
 - DeepSeek [1]
- 支持的训练算法包括但不限于：
 - (增量) 预训练
 - (多模态) 指令监督微调
 - 奖励模型训练
 - PPO 训练

- DPO 训练
- KTO 训练
- ORPO 训练
- 支持的运算精度有：
 - 16 bit 全参数微调
 - 冻结微调
 - LoRA 微调
 - 基于 AQLM/AWQ/GPTQ/LLM.int8/HQQ/EETQ 的 2/3/4/5/6/8 bit QLoRA 微调
- 支持的优化算法有：
 - GaLore
 - BAdam
 - DoRA
 - LongLoRA
 - LLaMA Pro
 - Mixture-of-Depths
 - LoRA+
 - LoftQ
 - PiSSA
- 支持的加速算子有：
 - FlashAttention-2
 - Unsloth
- 支持的推理引擎有：
 - Transformers
 - vLLM
- 支持的实验面板包括但不限于：
 - LlamaBoard
 - TensorBoard
 - Wandb
 - MLFlow

[1] 天数智芯针对 LLaMA、Qwen 和 DeepSeek 等系列大模型的全量和 LoRA 微调添加了 Megatron-LM 后端的支持，即您可以在天数智算软件栈上通过 LLaMA-Factory 调用 Megatron-LM 对相关大模型实现 3D 并行的微调训练，从而提升训练性能和效率。

目前 Megatron-LM 后端已支持全量和 LoRA 微调优化方式。

1.4 安装天数智芯适配版 LLaMA-Factory

天数智芯支持您通过以下两种方式安装 LLaMA-Factory：

- 通过 Docker 镜像安装 LLaMA-Factory
- 在宿主机上安装 LLaMA-Factory

1.4.1 通过 Docker 镜像安装 LLaMA-Factory

前置条件

生成包含天数智算软件栈的 Docker 镜像，详情请参考《软件栈安装指南》。

安装步骤

1. Docker 镜像中已包含天数智芯适配版 LLaMA-Factory，无需您手动安装。
2. 安装以下依赖：

```
# 安装天数适配版 accelerate
$ pip3 install accelerate-{version}+corex.{v.r.m}-py3-none-any.whl

# 安装天数适配版 transformers
$ pip3 install transformers-{version}+corex.{v.r.m}-py3-none-any.whl

# 安装开源版本 peft
$ pip3 install peft==0.11.1
```

3. 检查是否存在依赖问题，如果无输出，表示不存在依赖问题：

```
$ pip3 check | grep llamafactory
```

1.4.2 在宿主机上安装 LLaMA-Factory

前置条件

宿主机上安装天数智算软件栈，详情请参考《软件栈安装指南》。

安装步骤

1. 安装天数智芯适配版 LLaMA-Factory：

```
$ pip3 install llamafactory-{version}+corex.{v.r.m}-py3-none-any.whl
```

2. 安装以下依赖：

```
$ pip3 install accelerate-{version}+corex.{v.r.m}-py3-none-any.whl
$ pip3 install transformers-{version}+corex.{v.r.m}-py3-none-any.whl
```

1.5 LLaMA-Factory 使用示例

以下配置以调用 Megatron-LM 后端进行 LLaMA3-8B SFT 全量微调为例进行说明，请您根据实际情况调整相关配置。

1. 在任意目录下准备 Megatron-LM 相关的 megatron_config.json 文件，将下述配置写入该文件中 (该文件主要用于配置 Megatron-LM 训练的参数)：

```
{
  "tokenizer_model": "./data/Meta-Llama-3-8B-SFT/Meta-Llama-3-8B-Instruct/
    ↪ tokenizer.model",
  "tokenizer_type": "Llama3Tokenizer",
  "lr": 5e-5,
  "min_lr": 5e-6,
  "bf16": true,
  "lr_warmup_iters": 0,
  "normalization": "RMSNorm",
  "use_flash_attn": true,
  "swiglu": true,
  "use_rotary_position_embeddings": true,
  "untie_embeddings_and_output_weights": true,
  "add_bias_linear": false,
  "add_position_embedding": false,
  "bias_gelu_fusion": false,
  "log_interval": 1,
  "timing_log_level": 1,
  "variable_seq_lengths": true,
  "finetune": true,
  "global_batch_size": 1024,
  "tensor_model_parallel_size": 1,
  "pipeline_model_parallel_size": 4,
  "sequence_parallel": true,
  "use_distributed_optimizer": true,
  "recompute_granularity": "full",
  "recompute_method": "uniform",
  "recompute_num_layers": 1,
  "recompute_method_per_stage": [4,1],
  "recompute_num_layers_per_stage": [4,0],
  "num_layers_per_stage": [1,7,2,9,1,7],
  "check_for_nan_in_loss_and_grad": false,
  "create_attention_mask_in_dataloader": false
}
```

天数智芯特有参数的使用说明，可以参考《Megatron-LM 框架功能说明》；其它参数请参考 [Megatron-LM 官方说明](#)。

2. 在相同目录下准备 LLaMA-Factory 训练配置的 llama_factory_config.yaml 文件，将下述配置写入该文件中 (该文件包含 LLaMA-Factory 启动的相关配置，详情以及更多模型训练配置请参考 [LLaMA-Factory 训练配置说明](#))：

```
# 待训练模型
model_name_or_path: data/Meta-Llama-3-8B-SFT/Meta-Llama-3-8B-Instruct

# 训练方式
stage: sft
do_train: true
finetuning_type: full # lora

# 配置 megatron_config 文件路径，该文件为步骤 1 中准备的 json 文件
```

```
megatron_lm: sft_configs/megatron_lm/megatron_config_llama3_8b.json
```

```
# 待训练数据集
```

```
dataset: alpaca_gpt4_en
```

```
template: default
```

```
cutoff_len: 1024
```

```
max_samples: 10000
```

```
overwrite_cache: true
```

```
preprocessing_num_workers: 16
```

```
# 输出相关配置
```

```
output_dir: saves/llama3-8b/full/sft
```

```
logging_steps: 1
```

```
save_steps: 5000
```

```
plot_loss: true
```

```
overwrite_output_dir: true
```

```
# 训练相关配置
```

```
per_device_train_batch_size: 4
```

```
gradient_accumulation_steps: 1
```

```
learning_rate: 1.0e-5
```

```
num_train_epochs: 1.0
```

```
lr_scheduler_type: cosine
```

```
warmup_ratio: 0.1
```

```
bf16: true
```

```
ddp_timeout: 180000000
```

```
flash_attn: fa2
```

```
# 返回执行结果
```

```
val_size: 0.1
```

```
per_device_eval_batch_size: 1
```

```
eval_strategy: steps
```

```
eval_steps: 5000
```

3. 以在单机 8 卡天垓 150 环境上训练 LLaMA3-8B 为例，开始训练模型。在相同目录下新建一个 .sh 脚本，将下属配置写入该脚本中并运行：

```
# 安装 LLaMA3 依赖库
```

```
if ! python3 -c "import llama" &> /dev/null; then
```

```
    pip3 install git+https://github.com/meta-llama/llama3
```

```
fi
```

```
# 配置天垓 150 8 卡服务器训练相关参数
```

```
GPUS_PER_NODE=16
```

```
NNODES=$1
```

```
NODE_RANK=$2
```

```
MASTER_ADDR=$3
```

```
MASTER_PORT=12345
```

```
LOG_DIR="logs_$(date +%m%d%H%M%S)"
```

```
mkdir -p ${LOG_DIR}

export CUDA_DEVICE_MAX_CONNECTIONS=1
export NCCL_NET_SHARED_BUFFERS=0
export NCCL_ALGO=Ring
export OMP_NUM_THREADS=4
export NCCL_FORCESYNC_DISABLE=1
export NCCL_USE_HIGH_PRIORITY_WARP=1
export NCCL_PRIM_P2P_LEVEL=0
export WANDB_DISABLED=TRUE

FORCE_TORCHRUN=1 NNODES=${NNODES} RANK=${NODE_RANK} MASTER_ADDR=${MASTER_ADDR}
↪ MASTER_PORT=${MASTER_PORT} llamafactory-cli train llama_factory_config.yaml 2>&1
↪ | tee ${LOG_DIR}/train_rank${NODE_RANK}.log
```

1.6 常见问题

问题 1: 4 机环境下对大模型进行 SFT 全量微调并使用 ds_z3_offload_config.json 配置, 出现报错 launcher.py FAILED, exitcode 显示 -9。

解决方法

exitcode 显示 -9, 表示主机内存不足导致进程被 kill。在大模型训练时如果使用 ds_z3_offload_config.json 配置, 为了确保主机内存够用, 建议将主节点的主机内存配置为 1TB。

问题 2: 在 Docker 镜像中使用 LLaMA-Factory 训练, 提示 peft 版本不对 ImportError: peft>=0.11.1, <=0.12.0 is required for a normal functioning of this module, but found peft==0.10.0

解决方法

LLaMA-Factory 和 TGI 都依赖 peft, 但依赖版本不同。在生成 Docker 镜像时, TGI 的安装顺序后于 LLaMA-Factory, 因此 peft 被重装, 导致 LLaMA-Factory 依赖的 peft 版本不可用, 需要您再手动安装一次 (安装开源版本即可), 参考步骤如下:

```
# 安装开源版本 peft
$ pip3 install peft==0.11.1

# 检查 LLaMA-Factory 是否存在依赖问题
$ pip3 check | grep llamafactory
```

问题 3: 在 Docker 镜像中使用 LLaMA-Factory 训练, 提示环境内未包含适配的 accelerate 库 ImportError: Megatron is not installed. please build it from source

解决方法

由于 Docker 镜像中安装了 TGI, 会自动安装 accelerate, 导致原有的 accelerate 不可用, 需要您再手动安装一次 (需安装天数适配版), 参考步骤如下:

```
$ pip3 install accelerate-0.34.2+corex.{v.r.m}-py3-none-any.whl
```

此文档仅供【hoppygirl@163.com-云客】查阅，请勿分享他人

商标声明

- 天数智芯、天数智芯 logo、Iluvatar CoreX 等商标、标识、组合商标为上海天数智芯半导体有限公司之注册商标或商标，受法律保护。
- CentOS 标识为 Red Hat 公司的商标。
- Docker 为 Docker 公司在美国和其他国家的商标或注册商标。
- Linux 为 Linus Torvalds 在美国和其它国家的注册商标。
- NVIDIA、CUDA、GeForce 和 GeForce GTX 为 NVIDIA 公司在美国和/或其它国家的商标和/或注册商标。
- PyTorch 为 Facebook 公司的商标。
- TensorFlow 为 Google 公司的商标。
- Ubuntu 为 Canonical 公司的注册商标。

除上述明确列出的商标外，本内容中可能存在的任何其他未明示的产品名称、服务名称、企业名称、标识、口号、域名或其他商业标记 (无论是否标注 ™ 或 ® 符号)，均假定归属于其各自合法的商标权人所有。